Lab 4: Git Hooks, Bisecting, and Graphviz

Git Hooks

Git hooks are custom scripts that Git runs automatically at specific points in a repository's workflow. They allow you to automate tasks, enforce rules, or integrate tools like code checkers (linters) or tests directly into your Git process. We are focusing on client-side hooks. These are running on the user's machine and get triggered when a certain actions like committing gets executed.

Example:

- **pre-commit:** Runs before a commit is created (e.g., to check code).
- **post-commit:** Runs after a commit is completed (e.g., to send notifications).

Find Git hooks in the `.git/hooks/` directory of each repository. When you initialize a repository, Git provides example hook files (e.g., `pre-commit.sample`). When looking at this file, you'll notice that these are basic shell scripts that get executed (in this case) before a commit finishes. To enable such a script, make it executable:

```
chmod +x .git/hooks/pre-commit
```

We want to write our own `pre-commit` hook that checks our shell scripts for errors when we commit them. The commit gets aborted when errors occur. That way, we ensure that working scripts end up in the repository to increase our code quality.

A popular check program is `shellcheck` and available on our development machine. When passing a script with errors in it, `shellcheck` will report the errors along with hints on how to fix them.

Example:

```
cat script_with_errors.sh
#/bin/sh
```

Running `shellcheck` on the script returns:

```
shellcheck script_with_errors.sh
In script_with_errors.sh line 1:
#/bin/sh
```

```
4   ^-- SC1113 (error): Use #!, not just #, for the shebang.
```

For more information: https://www.shellcheck.net/wiki/SC1113

Assignment:

Write a `pre-commit` hook that calls `shellcheck` on those files in the commit that end with `.sh`. It should abort commits when shellcheck's return code is greater than zero. Echo the name of the file that caused the commit to abort to help with debugging. Don't forget to make the hook executable to activate it. To help debug your hook during a commit execution, you can activate git's debugging like this:

```
1 GIT_TRACE=1 git commit -m "Test commit"
```

This will give more details about the steps git executes during a commit, including your hook. You can find information about how to access the committed files from the example files in the `hooks` directory and from the git-commit man page.

File Finder

Write a shell script called `filefinder.sh` that has the following syntax:

```
1 filefinder.sh path extension
```

The first parameter `path` is the directory to search recursively for files that have the second parameter `extension`. Count the number of files found and echo the result to the screen. If there are no matching files with that extension, return that to the user. Only extensions count, even if the given extension (i.e. `txt`) appears as part of the filename. For example, `filefinder.sh . txt` does **not** match on a file called `txtfile`, but one with`myfile.txt` does match.

The script should check for the following errors:

- Path does not exist
- Path is a file
- The extension must not be empty

Commit the script into the Lab4 directory of your repository when finished.

Git Bisect

Scenario: The UFDcoders.com startup is asking you for help with a problem they have. They have been building a `script.sh` for a client and have made about 100 commits to it until they realized that the base functionality does not work anymore. They have no idea which of these 100 commits introduced the error (called regression). They can't simply start from scratch as each change has added some functionality that they do not want to lose. They need to find a way to identify the commit but do not want to go through each individual commit as that would take too long (the client is waiting). They provided you with their full commit history in the `UFDcoders.com` subdirectory of `Lab4`.

Assignment:

Familiarize yourself with the `git bisect` command: https://git-scm.com/docs/git-bisect

Then use `git bisect` on the `UFDcoders.com` program as follows. Find a commit that is broken, likely the latest one. Run the `script.sh` and see if the output is correct or not. Then mark the commit as `good` or `bad` according to the git bisect documentation. The `bisect` will present you with a different commit in the history and you repeat the process of checking the `script.sh` output. Do this until you find the problematic commit. You do not need to modify any of the commits in the `UFDcoders.com` directory.

At the end, provide the following:

- Name of the person who did the commit
- the commit hash
- the commit message

Put this into your `lab4-docs.md` file and commit it to your repository.

Data Plotting

Gnuplot is a terminal program to plot data in various ways. It can be used interactively or using a configuration file. We will use the latter to extract data from a CSV file that will serve as our data source (poor man's database).

First, familiarize yourself with the basic gnuplot syntax. You don't have to learn much to be able to create basic graphs that can visualize your data. Once you are more versed, you can create more fancy graphs (but this is beyond what we need to achieve for this lab exercise). Here is a tutorial link that should get you started (of course, you can also find other good tutorials on the Internet):

https://www.ibm.com/developerworks/aix/library/au-gnuplot/

Create a basic configuration file that can be used by Gnuplot to create a basic graph. Once that works, look at the data in the provided `population.csv` file, which is world population data. The file format is as follows:

```
Country Name,Country Code,Year,Value
Arab World,ARB,1960,93485943
...
```

The goal for this lab exercise is to visualize the population growth for a country over time. A shell script asks the user for the country code (i.e. ARB) and range in years (i.e. 1965 till 1980). You decide whether the program should use a `dialog`-based interface or simple command line interactions only. The program extracts the data from `population.csv` for the specified region and year range and puts it into a new file. Make sure to properly format that file so that Gnuplot can process it (i.e. replace commas with spaces). Use Gnuplot to plot the selected data as a PNG file based on your configuration file and the following requirements:

- Both the y- and x-axes should be labelled properly
- The graph should have a proper title
- Of course, the plot should correspond to the selected data for that country
- Use proper error checking within your script to check for invalid inputs

Note: don't get too fancy with the graph output. An ugly graph with the proper data is (arguably) more valuable than a pretty plot with the wrong data.

Make sure to clean up any temporary files created during the processing.